

微服务接口与数据归属

可编辑源为同名 Markdown。模型源：02_docs/models。渲染图：02_docs/images。

微服务划分、服务接口与数据归属方案

1. 实现范围

后端已经按业务能力拆分为 3 个可独立构建、测试和部署的业务微服务。API Gateway 只负责统一入口，不计入业务服务数量。微服务版本不再通过单体后端读写业务表，前端可通过网关完成 UC01-UC12。

业务服务 | 业务职责 | 独立数据库 | 代码位置

user-service | 注册登录、资料、密码、地址、角色和信用 | `softw\_users` | `services/user-service/`

product-service | 商品、二手、店铺、评价、聊天、议价、图片和库存 | `softw\_catalog` | `services/product-service/`

order-service | 下单、支付、取消、发货、收货、订单查询和状态机 | `softw\_orders` | `services/order-service/`

API Gateway | 路由、认证头透传、超时、CORS 和 503 降级 | 无 | `services/api-gateway/`

1.1 划分理由

1. 用户域围绕账号生命周期和收货身份，安全规则、密码和信用数据内聚。

2.

商品域围绕卖家的可交易资源，店铺资格、库存、评价和围绕商品的会话变化频率接近，统一维护商品一致性。

3. 订单域围绕不可逆交易状态和历史快照，独立保存订单，不通过跨库关联重建历史价格、地址或商品名称。

4. 不是按 9 个用例机械拆成 9 个服务。三个边界分别对应身份、交易标的和交易过程。

2. 服务划分图

【图文件见仓库：images/33-MS-SPLIT.svg。矢量图在 02_docs/images 与 models。】

可编辑 Mermaid 源文件：33-MS-SPLIT.mmd

图中的三个数据库可以运行在同一个 MySQL 8 服务器，但数据库名、连接配置和 ORM 模型均按服务隔离。任何服务都没有其他服务数据库的模型或跨库联表代码。

3. 服务接口清单

3.1 API Gateway

路径前缀 | 目标服务

`/api/users`, `/api/addresses` | user-service

`/api/products`, `/api/secondhand`, `/api/shops` | product-service

`/api/evaluations`, `/api/chats`, `/api/uploads`, `/uploads` | product-service

`/api/orders` | order-service

网关和三个业务服务统一提供 /health (存活)、/ready (依赖就绪) 和 /version (版本、提交 SHA、构建标识)。网关转发 JWT、Content-Type 和 Idempotency-Key。依赖服务超时或断开时返回 HTTP 503，不返回伪造业务结果。

3.2 user-service

方法 | 路径 | 业务目标

POST | `/api/users/register` | 校验并创建用户，bcrypt 保存密码，签发 JWT

POST | `/api/users/login` | 校验密码并签发 JWT

GET/PUT | `/api/users/profile` | 查询、修改当前用户资料

PUT | `/api/users/password` | 校验旧密码后修改密码

GET/PUT | `/api/addresses` | 查询或整体替换当前用户地址，保证唯一默认项

GET | `/internal/users/:id` | 向业务服务提供最小用户快照

POST | `/internal/users/:id/credit` | 接收评价或履约信用变化

POST | `/internal/users/:id/role` | 店铺认证后更新卖家角色

3.3 product-service

方法 | 路径 | 业务目标

GET/POST | `/api/products` | 筛选分页或发布商品

GET/PUT/DELETE | `/api/products/:id` | 商品详情和所有者维护

GET | `/api/products/search`, `/api/products/recommended`, `/api/products/mine` | 搜索、推荐和本人商品

GET/POST/PUT/DELETE | `/api/secondhand/\*\*` | 二手商品完整生命周期

GET/PUT/POST | `/api/shops/\*\*` | 店铺查询、资料和演示认证

GET/POST/PUT | `/api/evaluations/\*\*` | 评价、评价查询和回复

GET/POST/PUT | `/api/chats/\*\*` | 会话、消息、议价和处理结果

POST | `/api/uploads/images` | 图片白名单上传

POST | `/internal/products/reservations` | 幂等校验商品并预留库存，返回订单快照

POST | `/internal/products/reservations/:id/release` | 取消或失败时幂等恢复库存

POST | `/internal/products/reservations/:id/complete` | 收货后完成预留，二手商品转为已售出

3.4 order-service

方法 | 路径 | 业务目标

POST/GET | `/api/orders` | 通过商品服务预留库存后创建订单，或查询买家订单

GET | `/api/orders/seller` | 按订单快照查询卖家订单

GET | `/api/orders/:id` | 查询订单参与者可见详情

POST | `/api/orders/:id/pay` | 待付款转待发货

POST | `/api/orders/:id/cancel` | 取消订单并调用商品服务补偿库存

POST | `/api/orders/:id/ship` | 卖家填写物流并转待收货

POST | `/api/orders/:id/confirm` | 买家收货并完成商品预留

PUT | `/api/orders/:id` | 兼容按状态发货或确认收货，内部仍走库存完成接口

GET | `/internal/orders/\*\*/purchases/\*\*` | 向评价和退款流程证明购买关系

4. 数据表归属表

数据库 | 表 | 唯一管理服务 | 其他服务如何使用

`softw\_users` | `Users` | user-service | JWT 中只传用户 ID；通过内部用户接口获取快照或更新信用

`softw\_users` | `Addresses` | user-service | 订单接收前端提交的地址快照，不查询地址表

`softw\_catalog` | `Products` | product-service | 订单通过库存预留接口获取商品快照

`softw\_catalog` | `Shops` | product-service | 用户 ID 作为业务引用，不建立跨库外键

`softw\_catalog` | `Evaluations` | product-service | 通过订单内部接口验证已完成交易

`softw\_catalog` | `ChatConversations`, `ChatMessages` | product-service | 用户使用快照；退款资格通过订单接口验证

`softw\_catalog` | `InventoryReservations` | product-service | 订单只保存 `reservationId`，不能直接修改库存记录

`softw\_orders` | `Orders` | order-service | 商品和用户服务只通过购买证明接口读取必要结果

文件卷 | `uploads/` | product-service | 其他服务只保存或传递 URL

三个服务代码中没有跨服务 Sequelize association。订单的 items、shippingAddress 和物流信息是创建时快照，避免跨库联表。

5. 跨服务调用与失败处理

调用 | 成功行为 | 失败处理

order-service -> product-service: 库存预留 | 原子锁定商品行、扣减库存、写入幂等预留记录 |

商品不存在、不可售或库存不足时不创建订单；超时返回 503

order-service -> product-service: 库存释放 | 取消订单或订单落库失败后恢复库存 | 释放接口按 `reservationId` 幂等，重复调用不重复增加库存

order-service -> product-service: 交易完成 | 将预留标记完成，二手零库存商品转已售出 | 调用失败时订单不进入已完成，允许重试

product-service -> order-service: 评价/退款资格 | 校验买家、商品和订单状态 | 校验失败不创建评价或退款申请

product/order-service -> user-service: 身份 | 读取最小用户快照 | 用户服务不可用时拒绝需要身份的写操作；公开商品查询仍可用

product/order-service -> user-service: 信用 | 按评价或履约结果更新信用 | 非核心信用更新失败不回滚已完成物流，记录为可重试调用

Gateway -> 业务服务 | 3 秒内返回真实上游结果 | 超时或连接失败返回 503，并注明失败路由

内部接口必须携带

X-Internal-Token。外部客户端只能通过网关和公开业务接口访问，不能调用库存、信用和购买证明接口。

6. 独立构建、测试与部署证据

验收项 | 证据

独立构建 | 三个服务各自拥有 `package.json`、锁文件和 Dockerfile

独立测试 | `services/\*\*/app.test.js` 使用真实 MySQL 验证各自数据库和业务规则

Compose | `03\_devops/docker-compose.microservices.yml` 启动 MySQL、3 个业务服务、网关和前端

Kubernetes | `03\_devops/k8s/microservices/` 包含 MySQL PVC、三个 Deployment/Service、网关、前端和 HPA

CI 门禁 | `.github/workflows/ci-cd.yml` 的微服务矩阵测试成功后才允许构建镜像和部署

公开 API/E2E | `04\_tests/microservices/public-api-manifest.js` 与 `api-e2e.test.js` 从网关覆盖 49 项公开业务接口和 UC01-UC12；实测 15/15 通过，清单见 `微服务公开API测试映射.md`
可观测性 | `npm run k8s:observe` 查看 Pod、探针、SHA 版本、日志和 Warning Events
失败诊断 | 流水线失败自动上传 describe、Events 和容器日志；本地错误镜像实验已实际恢复
页面回归 | Compose 微服务前端 `http://localhost:8082` 已运行 Playwright UC01-UC12，6/6 通过
Kind 实测 | `04\_tests/reports/kubernetes-deployment/2026-09-02-audit/` 记录 7 个带版本标签镜像的真实滚动部署、健康/就绪/版本检查

公开路由清单、UC01-UC12 的 MAIN/ALT/ERR 分支、测试编号和自动防漂移规则统一维护在 02docs/微服务公开API测试映射.md。新增公开路由但未同步清单、文档或网关 E2E 时，
npm run test:services:inventory 会失败并阻断镜像构建与部署。

7. 改造前后版本

- 改造前：Git 标签 monolith-start，固定保留单体版本。
- 改造后：本次实现包含 services/、网关、隔离数据库和部署配置；提交并通过远程流水线后打 microservices-v1 标签。

本地启动：

```
docker compose --env-file .env -f 03_devops/docker-compose.microservices.yml up -d --build --wait
curl --fail http://127.0.0.1:8081/health
curl --fail http://127.0.0.1:8081/ready
curl --fail http://127.0.0.1:8081/version
open http://localhost:8082/
```